

## SFC-RG JOINT LECTURE

# Threat Clarification and Correctness Proofs in Protocol Design

---

Graduate School of Media and Governance, Keio University

Riku MOCHIZUKI

moz@sfc.keio.ac.jp

May 1, 2025

# Topics and Objectives

## Topics

- Introduction to protocol design
- Introduction to threat models
- Introduction to formal proofs of protocol correctness
- Introduction to research challenges in distributed systems and digital identity

## Objectives

- Understand the principles of protocol design in distributed systems
- Understand the approach to correctness guarantees in protocol design
- Understand how to structure academic papers about protocols and what aspects need to be clarified

# What is a Protocol?

## Definition of Protocol (Oxford Dictionary)

A set of rules governing the exchange or transmission of data between entities.

**Entity:** Refers to individual participants (users, devices) in a protocol.

## Definition of Algorithm (Oxford Dictionary)

A set of instructions for performing a task or solving a problem.

While algorithms focus on computational procedures, protocols emphasize interactions between participants. <sup>1</sup> Common protocol examples: HTTP, SMTP, TLS/SSL, SSH, OAuth, Paxos, Raft

---

<sup>1</sup>Protocols are often explained as conversing using common rules like English or Japanese, which represents interaction rules. The term “Japanese algorithm” is not common because algorithms represent computational procedures rather than interactions.

# How to Design Correct Protocols

This lecture does not cover protocol design methodologies.

This lecture focuses on how to explain and demonstrate the correctness of protocols you design in your domain

To demonstrate protocol correctness, the following aspects must be clarified:

- Which entities participate in the protocol
- What information exchanges occur between entities
- What conditions must be satisfied for the protocol to be achieved the goal
- **What behaviors each entity might exhibit**
- **How to determine whether the protocol is correct<sup>2</sup> given these entity behaviors**

---

<sup>2</sup>Satisfying both Safety (bad things don't happen) and Liveness (good things eventually happen) properties

# Adversaries and Threat Models

In protocol design, clearly defining assumptions about entity capabilities and behaviors is crucial when considering what behaviors to allow.

## Adversary (Oxford Dictionary)

An adversary refers to an attacker, typically with malicious intent, who undertakes actions against a system or protocol

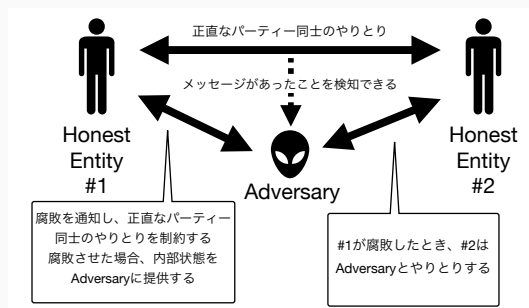
In protocol design, an adversary can control entities participating in the protocol, modifying or monitoring their operations.

## Threat Model

A threat model formalizes assumptions about adversary capabilities and behaviors.

# Adversaries and Threat Models

Adversaries control honest entities and introduce corrupted entities into the protocol.



**Figure 1:** Relationship between adversaries and entities. Adversaries can introduce corrupted entities into the protocol by intercepting messages from honest entities or directly controlling entities. (Note that entities do not become corrupted directly!)

# Components of Threat Models

Threat models consist of 5 key components:

1. **Corruption Model.** Behavior patterns of entities controlled by the adversary
2. **Computational Power.** Constraints on computational resources available to the adversary
3. **Adaptivity.** Timing and methods by which the adversary corrupts entities
4. **Visibility.** Range of information observable by the adversary
5. **Mobility.** Patterns of entity corruption and recovery by the adversary

Different combinations of these components define different threat models. The selection of components depends on the protocol's purpose and the environments in which it may be executed.

# Corruption Models

Corruption models define how adversaries can control entities.

- **Passive.** Follows the protocol completely while accumulating and utilizing all observed information. Cannot deviate from protocol specifications.<sup>34</sup>
- **Crash.** Can stop responding at any time but operates according to the protocol outside of downtime.
- **Omission.** Entities can arbitrarily delete or ignore received messages but cannot modify the content of messages they send.
- **Byzantine.** Entities can arbitrarily deviate from protocol specifications.

Each model encompasses the previous models: Byzantine  $\supset$  Omission  $\supset$  Crash  $\supset$  Passive

<sup>3</sup>For example, a bank employee who correctly processes customer transaction data while secretly recording and using that data for personal purposes.

<sup>4</sup>Also known as Honest-but-Curious or Semi-Honest model.



# Adaptivity Models

Adaptivity models define when and how adversaries corrupt participants:

- **Static:** Adversaries select entities to corrupt before protocol execution and cannot change their selection during execution.
- **Adaptive:** Adversaries can dynamically select entities to corrupt during protocol execution.

Consider the Adaptive model when there are few entities, many communication phases in the protocol, or long protocol execution times. For example, key exchange models typically consider the Static model. Consensus algorithms typically consider the Adaptive model.

# Visibility Models

Visibility models define the extent to which adversaries can observe messages and internal states.

- **Full Information.** Adversaries can completely observe all message contents and internal states (execution variables) of each entity
- **Private Channels.** Communication content between honest entities is completely secret; adversaries can know that messages are being sent and received but cannot see their contents. Internal states of honest entities are also hidden from adversaries.
- **Erasure Model.** Extends Private Channels by adding the constraint that adversaries cannot observe or recover data erased before an entity becomes corrupted.
- **Oblivious.** Message header information (sender, recipient, length) is observable by adversaries, but message content (payload) is completely hidden. The existence of communication and its metadata are exposed, but the content is protected.<sup>5</sup>

---

<sup>5</sup>This concept is used in specific encrypted communication protocols and anonymous communication systems.

# Computational Power Models

Computational power models define limitations on adversaries' computational resources:

- **Unbounded.** Adversaries with unlimited computational resources. Theoretically capable of breaking all cryptography.
- **Polynomially Bounded.** Adversaries limited to polynomial-time computation<sup>6</sup>
- **Fine-grained.** Adversaries with specific computational limitations.  
For example, models with limitations on hash computations per second (Hashrate)

The focus is on the adversary's computational power, not the entities' computational power. Adversaries can observe entity messages and internal states according to the Visibility Model. This model defines what computational resources adversaries can use to utilize that information.

<sup>6</sup>When solving a problem, the computational complexity can be expressed as order  $O(n^c)$  where  $n, c \in \mathbb{Z}$ . Examples include the difficulty of discrete logarithm problems and integer factorization problems. Generally, Probabilistic Polynomial Time (PPT) refers to algorithms that use randomness and always terminate within polynomial time  $\lambda^c$  relative to the security parameter  $\lambda \in \mathbb{Z}$  (e.g., key length)

# Mobility Models

Mobility models define patterns of entity corruption and recovery by adversaries:

- **Fixed.** Once corrupted, entities remain corrupted throughout protocol execution and cannot recover
- **Mobile.** Adversaries can dynamically corrupt and recover entities. However, there is an upper limit on the total number of corrupted entities at any given time<sup>7</sup>

From the perspective of IoT, mobile devices, and long-term system operation, the Mobile model is generally considered.

---

<sup>7</sup>Be careful not to confuse this with the Adaptive model in Adaptivity Models. Adaptive refers to the ability of adversaries to choose when to corrupt entities. Mobility considers whether corrupted entities can recover and become honest entities again under those conditions.

# Importance of Appropriate Assumptions

In security protocol research, clarifying assumptions is extremely important.

- If assumptions are too weak: Protocols may be vulnerable to realistic attacks
- If assumptions are too strong: Design and implementation may become complex, potentially reducing performance and practicality

Value of explicit assumptions:

- Enables accurate understanding of protocol correctness
- Allows fair comparison of different protocols using the same criteria

In actual system design, selecting appropriate assumptions (threat models) based on specific uses and potential risks is important.

## Example: Practical Byzantine Fault Tolerance (PBFT)

### PBFT

- PBFT is a replication protocol that **tolerates Byzantine faults** while achieving linearizability<sup>89</sup>
- This holds only when  $n \geq 3f + 1$  (where  $f$  is the number of adversaries and  $n$  is the total number of replicas).

The PBFT protocol was the first to demonstrate achieving linearizability in the partial synchrony model. Even 20 years after its introduction, PBFT remains widely used.

---

<sup>8</sup>Linearizable replication means all replicas process client requests in the same order, and changes are always reflected within the write request session. Each replicated data is assigned a sequence number and processed according to this sequence number.

<sup>9</sup>In Turing-complete state machines, replicated data is viewed as operations (state transition functions). When operations  $f$  and  $g$  exist, generally  $f \circ g \neq g \circ f$ , so execution order is important.

# PBFT Protocol Details (1)

Main phases of PBFT:

1. **Pre-prepare:** Primary (leader)  $p$  assigns sequence number  $s$  to client request  $r$  and sends  $\langle \text{PRE-PREPARE}, s, r \rangle$  message to other replicas
2. **Prepare:** Replicas agree on request order and send  $\langle \text{PREPARE}, s, r \rangle$  message to all replicas
3. **Commit:** Replicas that receive at least  $2f + 1$  PREPARE messages send  $\langle \text{COMMIT}, s, r \rangle$  message, committing to execution
4. **Reply:** Replicas that receive  $2f + 1$  COMMIT messages execute the request and send results to the client

## PBFT Protocol Details (2)

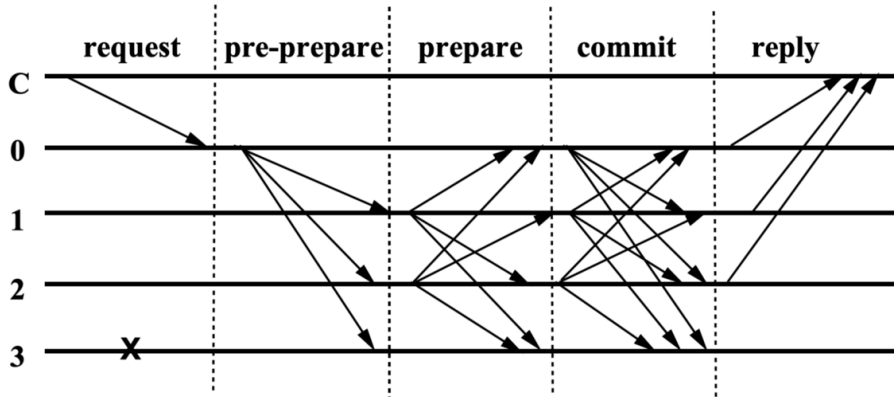


Figure 2: PBFT sequential diagram



# PBFT Threat Model

Let's consider the PBFT threat model.

## PBFT Threat Model $\Lambda_{PBFT}$

- **Corruption:** Byzantine - Replicas can behave arbitrarily, such as sending different messages to different replicas
- **Adaptivity:** Adaptive - Adversaries can dynamically corrupt different replicas in each phase of the protocol
- **Computational Power:** Polynomially Bounded - Limited to polynomial-time computation
- **Visibility:** Private channels - Communication content between honest replicas is secret, and internal states are not visible to adversaries
- **Mobility:** Fixed - Once corrupted, replicas remain corrupted throughout protocol execution and cannot recover

# PBFT Correctness Guarantees

## PBFT Correctness Guarantees

When  $n \geq 3f + 1$  (where  $f$  is the number of corrupted nodes and  $n$  is the total number of replicas):

- **Safety**: Honest replicas never commit different values for the same sequence number
- **Liveness**: All client requests are eventually processed and responses are returned

If we can prove that PBFT satisfies both Safety and Liveness, then PBFT can be considered correct.

In other words, if we can prove that PBFT satisfies Safety and Liveness regardless of what attacks adversaries perform within the threat model  $\Lambda_{PBFT}$ , then PBFT can be considered correct.

**Question:** Why is Safety satisfied when  $n \geq 3f + 1$ ? Let's explain why PBFT is secure!

# PBFT Safety Proof

Assumption: Two different values  $v$  and  $v'$  are committed with the same sequence number  $s$ .

## Lemma

*For each value to be committed, COMMIT messages from at least  $2f + 1$  replicas are required. (Out of  $3f + 1$  total replicas, at most  $f$  can be corrupted)*

- Value  $v$  requires COMMIT messages from at least  $2f + 1$  replicas in set  $\mathcal{P}$
- Similarly, value  $v'$  requires COMMIT messages from at least  $2f + 1$  replicas in set  $\mathcal{Q}$
- $|\mathcal{P} \cap \mathcal{Q}| \geq 2f + 1 + 2f + 1 - n \geq 4f + 2 - (3f + 1) = f + 1$
- The intersection  $\mathcal{P} \cap \mathcal{Q}$  contains at least  $f + 1$  replicas, and since the system has at most  $f$  Byzantine replicas,  
at least 1 honest replica must have sent COMMIT messages for both values  $v$  and  $v'$
- However, honest replicas never send COMMIT messages for different values with the same sequence number, creating a contradiction
- Therefore, the assumption is false

# Correctness Proofs for Complex Protocols

In practice, simple protocols like PBFT are rare, and composite protocols that combine existing protocols as components are common.

- Application layer protocols are often built on existing replication layers (e.g., PBFT) <sup>10</sup>

## Hybrid Protocol

A hybrid protocol obtained by combining  $n$  protocols  $\pi_1, \dots, \pi_n$  is denoted as  $\pi[\pi_1, \dots, \pi_n]$ .

**Motivation:** How to demonstrate the correctness of composite protocol  $\pi[\pi_1, \dots, \pi_n]$ ?

- Can we analyze each component individually to prove the correctness of the whole?
- How to guarantee correctness in any environment?

---

<sup>10</sup>Example: Building a timestamp mechanism on top of PBFT. Related to kekeho's GP2.

# Universal Composability Framework

Using the UC Framework, the correctness of complex protocols can be demonstrated based on the completeness guarantees of simple components. In other words, to prove that a composite protocol  $\pi[\pi_1, \dots, \pi_n]$  is correct, we need to prove the correctness of each component  $\pi_1, \dots, \pi_n$  and show that they are combined appropriately.

## Definition of Ideal Functionality $\mathcal{F}$

An ideal functionality  $\mathcal{F}$  receives inputs from entities, computes  $f$ , and returns outputs to each entity. In other words, it defines the protocol interface and defines a mapping  $f$  that returns outputs for any input.<sup>11</sup>

## Definition of Protocol $\pi$

A protocol  $\pi$  is an actual program, a procedure executed between multiple entities (concrete implementation of the protocol). It is designed to realize the ideal functionality  $\mathcal{F}$ .

---

<sup>11</sup>This can be considered as an indestructible (i.e., the interface or mapping  $f$  cannot be modified) trusted entity.

UC has two worlds for executing ideal functionality  $\mathcal{F}$  and protocol  $\pi$ :

## Ideal World

- The ideal functionality  $\mathcal{F}$  performs computations, and the adversary behaves as a simulator  $Sim$ .
- Dummy entities send inputs to  $\mathcal{F}$  and return outputs from  $\mathcal{F}$  to the environment  $\mathcal{Z}$ .
- The simulator  $Sim$  mimics the behavior of the real-world adversary through the interface with the ideal functionality  $\mathcal{F}$  and returns the results to the environment.

# Ideal/Real World Paradigm

UC has two worlds for executing ideal functionality  $\mathcal{F}$  and protocol  $\pi$ :

## Real World

- The real world consists of the actual protocol  $\pi$ , its participating entities, and the adversary  $\mathcal{A}$ .
- Entities communicate with each other according to protocol  $\pi$  while advancing the computation.
- The adversary  $\mathcal{A}$  controls corrupted entities, and all internal states and actions are controlled by  $\mathcal{A}$ .
- The environment  $\mathcal{Z}$  provides inputs to entities and the adversary and collects all outputs.

In other words... each world can be interpreted as follows:

- The ideal world can be considered as "a world where a trusted entity  $\mathcal{F}$  performs all processing."
- The real world can be considered as "a world where entities communicate with each other to perform processing because there are no trusted entities."
- Therefore, a protocol being correct means that the protocol  $\pi$  executed by entities in the real world is indistinguishable from the ideal functionality  $\mathcal{F}$  executed in the ideal world.<sup>a</sup>

---

<sup>a</sup>More precisely, it is indistinguishable with negligible probability. This is because UC holds only against probabilistic polynomial-time (PPT) adversaries. Regarding probabilistic polynomial time, as mentioned earlier, it can be approximated as order  $O(n^c)$  in a probabilistic Turing machine.



# Environment Entity Operating Ideal and Real Worlds

The entity that operates these two worlds is the environment  $\mathcal{Z}$ .

## Environment $\mathcal{Z}$

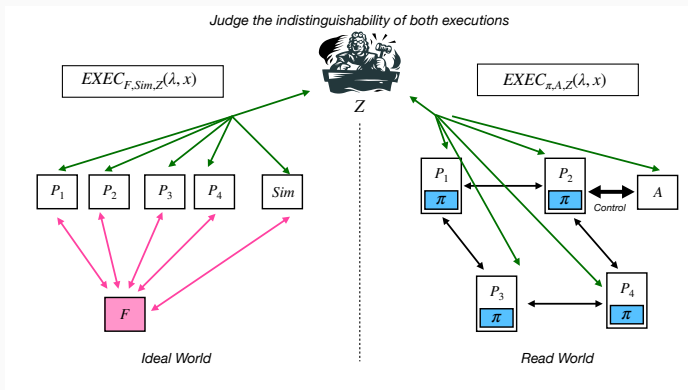
- The environment  $\mathcal{Z}$  abstracts all systems outside the protocol (such as other protocols).
- In the ideal world,  $\mathcal{Z}$  sends protocol inputs to dummy entities and receives their outputs and outputs from the simulator  $\mathcal{Sim}$ .<sup>12</sup>
- In the real world,  $\mathcal{Z}$  sends inputs to entities executing the protocol and receives their outputs and outputs from the adversary  $\mathcal{A}$ .
- $\mathcal{Z}$  tries to distinguish whether it is interacting with the ideal world or the real world.<sup>13</sup>

---

<sup>12</sup>More precisely, the environment does not communicate directly with  $\mathcal{F}$  but interacts indirectly through dummy entities

<sup>13</sup>In other words,  $\mathcal{Z}$  tries to find differences between the behavior of the actual protocol and entities in the real world and the behavior of the ideal functionality and dummy entities in the ideal world.

# Comparison of Ideal and Real Worlds



**Figure 3:** Interaction between ideal and real worlds. In the ideal world, each dummy party  $P_i$  communicates directly with the ideal functionality  $\mathcal{F}$ , while in the real world, entities  $P_i$  executing protocol  $\pi$  communicate with each other. This perspective allows  $\mathcal{F}$  to be considered as a trusted ideal functionality.

# Correctness Claims in UC

## Definition of UC Correctness

If the environment  $\mathcal{Z}$  tries to find differences between interactions in the real world and the ideal world but cannot distinguish between them, then  $\pi$  can be said to correctly realize the ideal functionality  $\mathcal{F}$ . This is called “ $\pi$  UC-realizes  $\mathcal{F}$ .”

**Rigorous Definition.** For security parameter  $\lambda \in \mathbb{N}$  and protocol  $\pi$ , we say that  $\pi$  “UC-realizes” the ideal functionality  $\mathcal{F}$  when: For any probabilistic polynomial-time<sup>14</sup> adversary  $\mathcal{A}$ , there exists a probabilistic polynomial-time algorithm  $Sim$  such that for any probabilistic polynomial-time environment  $\mathcal{Z}$  and any input  $x \in \{0, 1\}^*$ :

$$EXEC_{\mathcal{F}, Sim, \mathcal{Z}}(\lambda, x) \approx EXEC_{\pi, \mathcal{A}, \mathcal{Z}}(\lambda, x)$$

---

<sup>14</sup>UC holds only against probabilistic polynomial-time adversaries

## Benefits of Using UC for Correctness Proofs

- **Modularity.** Complex tasks can be decomposed into smaller protocols, and the correctness of each protocol can be proven independently.
- **Composition Theorem.** If protocol  $\pi$  UC-realizes ideal functionality  $\mathcal{F}$  and protocol  $\rho$  using  $\mathcal{F}$  UC-realizes ideal functionality  $\mathcal{G}$ , then the protocol  $\rho^{\mathcal{F} \rightarrow \pi}$  obtained by replacing  $\mathcal{F}$  with  $\pi$  also UC-realizes  $\mathcal{G}$ .

The core of the UC framework is "composability": the correctness of a protocol, once proven, is maintained in any complex environment. This enables the incremental construction and reuse of secure protocols.

# Protocol Instances and Session Identification

In the real world, multiple copies of the same protocol are executed simultaneously by different users. A protocol is like a type, and instances of that type are actually executed.

## Protocol Instance

An instance of protocol  $\pi$  is a specific execution of that protocol, typically distinguished by a unique identifier.

- **Session Identifier** ( $sid$ ): A unique identifier that distinguishes each protocol execution
- For example... on blockchains, smart contract addresses function as  $sid$

In the basic UC framework, each protocol instance has its own instance. However, this may not be realistic in some cases (e.g., when the same hash function is used in multiple protocols).

# Generalized Universal Composability (GUC) Framework

## What is the GUC Framework

An extension of the UC framework that introduces the concept of "global functionalities" shared across multiple protocol instances.

## Global Functionality

- Functionality shared across multiple protocol instances
- Examples: hash functions, digital signatures, shared ledgers
- Maintains consistent internal state across all protocol sessions

GUC not only enables parallel execution environments but also eliminates the need to analyze instances individually.

# Why Learn These Correctness Proof Methods?

To write academic papers in formats accepted by the academic community.<sup>a</sup> Without these methods, papers may be rejected outright even if the protocol design appears good.

<sup>a</sup>Especially for international journals and top conferences. Top conferences have acceptance rates of 20% or lower, or CORE rank A or A<sup>+</sup> conferences

Simply describing protocol specifications and implementations is insufficient

*“ Since there is no formal security model, it is hard to verify the formal correctness of the claims. Moreover, the scheme includes many low-level implementation details instead of giving a general description of their scheme. ”*

(Excerpt from a review comment received by moz two years ago from CRYPTO'23, a top conference)



International Association for Cryptologic Research (IACR)

<https://eprint.iacr.org> › ... PDF

頻繁にアクセス

**A New Paradigm for Cryptographic Protocols**

R. Canetti 著 · 2000 · 被引用数: 4519 — **Composability** (or, rather, **de-composability**) can be regained

# Let's Examine a Paper Published in a Top Conference

Let's examine a paper titled "FairSwap: How to Fairly Exchange Digital Goods" published in ACM CCS 2020, a top conference.

This paper proposes a protocol for exchanging digital data  $x$  and  $y$  between  $\mathcal{A}$  and  $\mathcal{B}$ . The protocol guarantees that either both conditions  $P : \mathcal{A} \xrightarrow{x} \mathcal{B}$  and  $Q : \mathcal{B} \xrightarrow{y} \mathcal{A}$  are satisfied, or neither is satisfied.

The paper defines the ideal functionality  $\mathcal{F}_{fe}$  along with the global functionality  $\mathcal{L}$  with blockchain capabilities and a random oracle  $\mathcal{H}$ .

<https://dl.acm.org/doi/pdf/10.1145/3243734.3243857>



## FairSwap: How to Fairly Exchange Digital Goods

Stefan Dziembowski  
Institute of Informatics  
University of Warsaw, Poland  
stefan.dziembowski@crypto.edu.pl

Lisa ECKEY  
Department of Computer Science  
TU Darmstadt, Germany  
lisa.eckey@cs.tu-darmstadt.de

Sebastian Faust  
Department of Computer Science  
TU Darmstadt, Germany  
sebastian.faust@cs.tu-darmstadt.de

### ABSTRACT

We introduce FairSwap – an efficient protocol for fair exchange

Unfortunately, it has been shown that without further assumptions it is impossible to design protocols that guarantee such strong fairness guarantees [90]. We propose to circumvent this impossibility



# Are These Assumptions Sufficient?

The assumptions we have used so far are:

- **Network Model:** Asynchronous, synchronous, or partially synchronous (with an upper bound on communication delay)
- **Adversary Model:** How adversaries behave
- **Number of Corrupted Nodes:** Conditions on the number of entities the attacker can control (e.g.,  $n \geq 3f + 1$ )

**Question:** Are these the only threats and assumptions to consider when operating in the real world?

# Sybil Attack

With just these assumptions, all protocols are vulnerable to Sybil Attacks.

## Sybil Attack

An attack where a single entity impersonates multiple entities to gain undue influence in a system.<sup>15</sup>

PBFT requires the condition  $n \geq 3f + 1$ , where  $n$  is the number of nodes and  $f$  is the number of adversaries. This implicitly assumes that each replica is uniquely identifiable and message origins are authentic. If an adversary can behave as multiple entities, this condition breaks down.<sup>a</sup>

<sup>a</sup>This allows certain compromises in the Byzantine Corruption Model. Is this acceptable?

<sup>15</sup>One entity plays the roles of two different identities, but other entities cannot determine whether these identities belong to the same entity.

# Digital Signatures

To verify the authenticity of message origins, messages must be digitally signed and these signatures must be verified.

## Digital Signatures

Digital signatures use mathematically related public key (verification key)  $pk$  and private key (signing key)  $sk$  pairs to verify message origins.

- Signing a message  $m$ :  $\sigma \leftarrow \text{SIGN}(sk, m)$
- Verifying a message  $m$  with signature  $\sigma$ :  $\{0, 1\} \leftarrow \text{VERIFY}(pk, m, \sigma)$  where 1 if  $m$  is valid, 0 otherwise

Specific digital signature schemes include RSA, ECDSA, EdDSA, and Schnorr signatures.

# Verifying Message Authenticity

When entity  $\mathcal{P}$  sends a message  $m$  to entity  $\mathcal{Q}$ , entity  $\mathcal{Q}$  needs to verify that message  $m$  was indeed received from  $\mathcal{P}$ .

- Entity  $\mathcal{P}$  generates a digital signature  $\sigma$  for message  $m$  using the private key  $sk_{\mathcal{P}}$  corresponding to public key  $pk_{\mathcal{P}}$ .
- $\mathcal{P}$  sends  $(m, \sigma)$  to  $\mathcal{Q}$ .
- $\mathcal{Q}$  verifies the received  $(m, \sigma)$ :  $\text{VERIFY}(pk_{\mathcal{P}}, m, \sigma) \stackrel{?}{=} 1$

Assumption needed to ensure verification validity:  $\mathcal{Q}$  knows  $\mathcal{P}$ 's public key  $pk_{\mathcal{P}}$ .

**Question:** Is this sufficient to prevent Sybil Attacks?

# Cost of Generating Identifiers

**Answer:** Insufficient. The proposition “Digital signatures can distinguish message origins  $\Leftrightarrow$  Entities can be distinguished” does not necessarily hold.

Digital signatures can distinguish message senders, but verification validity is not guaranteed unless a one-to-one correspondence between entities and public keys is assumed.

- A single entity can generate multiple public keys and behave as multiple entities
- A Key Management System<sup>16</sup> is necessary to ensure one-to-one correspondence between entities and public keys. KMS verifies this correspondence<sup>17</sup>

<sup>16</sup>Key Management System (KMS) guarantees one-to-one correspondence between entities and public keys. Public Key Infrastructure (PKI) is a type of Key Management System.

<sup>17</sup>Verification involves two aspects. First, verifying that an entity claiming to be a specific identity is indeed that identity. Second, defining a mapping between entities and public keys, and confirming this relationship is bijective. After confirmation, KMS issues a signature (=certificate) indicating the binding is valid. Users can substitute this verification with signature verification by trusting the KMS.

# Problems with KMS

However, KMS has the following issues:

- Allowing certification authorities = assuming the existence of trusted third parties
- Creates single points of failure and scalability problems

Other approaches like **Proof-of-Work** require computational resources for creating new identifiers.

- When examining protocols (with Byzantine Fault Tolerance) in the context of distributed systems, bottlenecks exist (implicit, non-trivial assumptions) that prevent maximizing the appeal and effectiveness of distributed systems. These are not always documented in academic papers.

# One more thing

How to find impactful problems?

- Having operational experience expands the options for interpreting protocols discussed in theory.
- This means changing the situation in which a protocol operates (assumptions supporting the logic of the proposed protocol<sup>a</sup>) based on insights gained from operations, and determining whether bottlenecks can be ignored or not.
- I believe such exploration contributes to impactful and clear problem statements.

---

<sup>a</sup>For example, adversary models, network, and timing models

This lecture provided a theoretical perspective, but hearing operational viewpoints from other kGs may bring new interpretations to your research (field). I hope this general lecture becomes a forum where we organically generate, share, and enhance such interpretations together.